

Action (Verb + Noun)	Plain Explanation	Evidence
	invocation time to bias its analytical perspective toward domain-specific patterns.	<i>agents are each given the full interview transcript as input" P27</i>
NEW Retain Role-Specific Perspective	Maintain a domain-expert lens through downstream processing steps to produce perspective-diverse outputs alongside identity-neutral agents.	<i>"others may retain role-specific perspectives to support diverse theme formulation" P27</i>

V2 (validated):

Verb + Noun format · with plain-English explanations

Differences between Learning actions, reasoning actions, and decision-making actions.

● Memory Actions

Action	Plain Explanation
Store information in working memory	Put whatever the agent is currently using — recent inputs, intermediate results — onto a "scratch pad" for quick access.

Store/update experiences in episodic memory	Save past events as "episodes" — which game was won, which plan failed, what happened step by step — like a personal diary.
Store knowledge in semantic memory	Save general facts about the world — "fire is hot", "404 means not found" — not tied to any specific event.

● Grounding Actions

Action	Plain Explanation
Process perceptual inputs into textual observations	Turn images, sounds, or sensor readings into text descriptions so a text-based LLM can understand the physical world.
Affect physical environments via robotic planners	Actually move things in the real world — pick up a cup, open a door — by sending language commands to a robot arm.
Convert images to text via VLMs	Feed a picture to a Vision-Language Model (like GPT-4V) and get back a text description of what's in it.
Accept instructions from humans	Receive a task or command from a human user, e.g. "book me a flight" or "translate this paragraph."
Ask for clarification from people	When something is unclear, proactively ask the human — "Which file did you mean?"
Entertain people	Chat casually, tell jokes, do role-play — provide emotional support or fun.
Conduct multi-agent debate	Have multiple agents argue different sides of the same question, using the debate to reach a better answer.
Solve tasks collaboratively across agents	Split a big job among multiple agents — one writes code, one tests, one documents — and combine the results.

Interact with games, APIs, and websites	Operate in digital environments: play Minecraft, call a weather API, click buttons on a webpage.
Execute code	Run a piece of code and get the output — e.g. run a Python script, execute a database query.
Augment external knowledge or computation	Call outside tools to fill knowledge gaps or handle heavy math — look up a database, use a calculator, etc.

● Retrieval Actions

Action	Plain Explanation
Load skills from the skill library	Find and load a ready-made code snippet from a pre-built library — e.g., grab the "chop tree" skill in Minecraft.
Retrieve events from episodic memory	Search the "diary" for relevant past experiences, ranked by how recent, important, and relevant they are.
Leverage documents for code generation	Look up reference docs (API docs, tutorials) and use that information to help write code.
Retrieve knowledge from semantic memory	Pull a general fact from the knowledge base — e.g., look up "what does HTTP 404 mean?"

● Reasoning Actions

Action	Plain Explanation	Evidence
Read from working memory	Check the "scratch pad" — grab the intermediate results and state information stored there.	
Summarize recent observations, trajectories	Condense a flood of recent information into a few key takeaways to avoid information overload.	

Distill insights from retrieved information	Extract the core conclusion from a pile of raw data — "so the real issue is X."	<i>For every binary score $z(q)_{i,j}$ from the judge, there is a corresponding explanation $e(q)_{i,j}$. P3</i>
Provide context for subsequent LLM calls	Package the reasoning results so far and feed them as background into the next LLM call for a better answer.	

● Learning Actions

Action	Plain Explanation
Store episodic trajectories	Save a full action sequence from start to finish — every step recorded — for later training or review.
Update parametric policy	Change the model's internal weights through training so it actually performs better on similar tasks — real learning.
Establish a non-parametric policy	Instead of changing weights, build a case library of past experiences and look up the closest match when a new problem arrives.
Update semantic memory with knowledge	Generalize from raw experiences into reusable facts and save them — e.g., after three failures, note "this API has a rate limit."
Reflect on failed episodes	Do a post-mortem on what went wrong, store the lessons learned, and avoid making the same mistake again.

Build semantic map via VLM	Use a Vision-Language Model to scan the environment and produce a labeled map — not just layout, but annotations like "door here" or "enemy there."
Update LLM parameters via SL/RL/RLHF	Adjust the large language model's weights using supervised learning, reinforcement learning, or human feedback to improve on a specific task.
Self-improve	Have the LLM produce multiple outputs, pick the best ones, and train on those — the model bootstraps its own improvement.
Utilize subtask-specific LLMs	Use smaller, specialized models for specific jobs — one for summarization, one for SQL — instead of one big model for everything.
Update the source code as procedural memory	Modify the agent's own source code — essentially patching itself to change how it behaves.
Infer instructions from input-output examples	Look at a few "input → output" examples, figure out the underlying rule, and use that rule as a prompt for future tasks.
Update reasoning via prompt update	Rewrite its own prompt template so the LLM thinks more clearly next time — learning a better way to reason.
Update grounding via code-based skills	Write or improve code snippets that interact with the outside world — e.g., learn a new way to navigate a webpage.
Maintain curriculum library	Keep a syllabus of mastered and upcoming skills, arranged by difficulty, so the agent learns in a sensible order.
Update retrieval procedures	Improve how the agent searches for information — e.g., better keyword strategies or smarter relevance ranking.
Update decision-making procedures	Improve the decision process itself — e.g., go from "pick randomly" to "score every option and pick the best."

- Decision-Making Actions

Action	Plain Explanation
Propose action candidates	Brainstorm "what could I do right now?" — generate one or more possible next moves.
Evaluate actions via heuristics, LLM, or learned values	Score each candidate: use rules of thumb, ask the LLM to judge, or use a trained value function to estimate how good each option is.
Simulate grounding feedback internally	Mentally rehearse an action before doing it — imagine "if I do X, what will the environment look like?" without actually executing.
Select actions via argmax, softmax, or majority vote	Pick one action to execute: take the highest-scored one (argmax), sample probabilistically (softmax), or let multiple evaluators vote.
Execute grounding actions, API calls, or learning actions	Finally, do it for real: either perform an external action (call an API, move a robot) or an internal one (update memory, fine-tune).